DS 570 Final Project Guidelines

The final project should be a complete end-to-end data science project implementation, combining all the aspects of data science practice we've discussed in class. It must contain the following aspects in reasonable detail:

- Data processing
- Data visualization
- Interactivity with dashboards
- Predictive machine learning methods
- Evaluation of ML models
- Version control with Git
- Containerization with Docker

In addition to the technical aspects, the project is expected to be *interesting* and have some degree of *novelty.* You can, for example, develop a project that:

- Collects publicly available mid-size data and presents it in an interesting way, predicting some relevant variable, and assessing the results.
- Apply your domain knowledge to develop a predictive model relevant to your work (be careful not to use proprietary or private data).

Do **not** develop a project that

- Is AI slop.
- Uses well-trodden data sets such as Iris or MNIST, slaps some run-of-the-mill algorithms on them and wraps everything in a dashboard.
- Consists of a static Jupyter notebook.
- Does not run completely and independently in a container.
- Uses proprietary or closed data (e.g. requiring a special account login). However, putting access information in the container and getting programmatic access to a public storage system (AWS, GDrive, Dropbox, …) is acceptable, as long as it does not require the user to log in.

In a nutshell: **You are expected to teach your classmates and your instructor something new.**

The project **must** have a dedicated GitHub repository. The repository should be publicly accessible, unless you have a good reason for privacy (in that case, you must provide access to all class members). The development history should span multiple weeks. Projects that are uploaded in a single commit will **not** be considered.

The project **must** have a Dockerfile that allows the project to be run in a container. Give clear instructions in the readme file about how to run it. Everybody in class should be able to (and encouraged to) build the image and run the application in Docker.

A project that does not run completely in a Docker container will not get a passing grade.

The app can download the training and testing data from cloud storage or an API. However, this step must be fully automatized and should not require user intervention. That is, we must not need to download the data on our local filesystem, nor must we be required to sign up for a service.

The application should be reasonably fast: It should not require downloading gigabytes of data, or very long training times. Building the image and running the app should not take longer than a few minutes.

You will present your project with a short (~10 minute) oral demonstration, after which you will answer questions about the details. You should display a good understanding of your subject.

Checklist

Crucial Aspects (fails if missing)

- The GitHub repository is available, complete, contains a Dockerfile and a README describing the project details.
- The application image can be built and run with Docker.

Version Control & Repository Quality

- The commit history spans multiple weeks and reflects an iterative development process (not a single or few bulk commits).
- Commit messages are meaningful and descriptive, not generic (e.g., "fix", "update").
- The repository is well-organized with a logical directory structure (e.g., separate folders for data ingestion, modeling, app, tests).
- A `requirements.txt`, `pyproject.toml`, or equivalent dependency file is present and consistent with the Dockerfile.

Reproducibility & Containerization

- The Docker image builds successfully from scratch with a single command and without manual intervention.
- Data is fetched automatically at runtime without requiring local files or user accounts.
- The application runs end-to-end inside the container without external dependencies not declared in the Dockerfile.
- The application runs in a reasonable time (image build + data fetch + app launch under ~5 minutes).

Data & Modeling Rigor

- The data source is clearly documented (origin, collection method, licensing or terms of use).
- Data cleaning and preprocessing steps are explicitly justified, not just applied mechanically.
- Feature engineering choices are explained and motivated by domain or exploratory analysis.

- The choice of ML algorithm(s) is justified relative to alternatives (e.g., why this model and not another).
- Evaluation metrics are appropriate for the problem type (e.g., not defaulting to accuracy for imbalanced classification).
- There is no evidence of data leakage between training and testing pipelines.
- Baseline model or naive benchmark is provided for comparison.
- Uncertainty or limitations of the model are acknowledged.

Visualization & Dashboard

- Visualizations are self-explanatory and appropriately labeled (titles, axes, legends, units).
- The dashboard presents both exploratory/descriptive views and model results in an integrated way.

Novelty & Intellectual Contribution

- The project addresses a question or problem that is non-obvious and not a rehash of a known tutorial.
- The student demonstrates understanding of the domain, not just the tools.
- The presentation teaches the audience something genuinely new (dataset, method, domain insight, or framing).

Presentation & Documentation

- The README contains clear, step-by-step instructions to build and run the project.
- The README describes the problem, data, methods, and results at a level accessible to a technical peer.
- During the oral demo, the student can explain design decisions and answer questions beyond surface-level descriptions.
- The student can identify weaknesses or potential improvements in their own work.